

8

TREE DATA STRUCTURE

8.1 Introduction

8.1.1 Terminology

- (i) Internal Node : Node with atleast 1 child.
- (ii) Leaf Node : Node without any child.
- (iii) Root Node : Only node that does not have any parent.
- (iv) Complete binary Tree : Binary tree in which nodes are inserted from left to right at every level and we can not insert at node at $(K+1)$ th level until all levels upto K are fully filled.
- (v) Full Binary Tree : A binary tree in which every internal node has exactly two children and all the leaf nodes are at some level.

Binary tree that contains maximum number of nodes.

- (vi) Strict Binary Tree : A binary tree in which every internal node has exactly two children.
- (vii) Complete K-ary tree : Tree in which every internal node has exactly K-children.

8.1.2 Mathematical Result

- (i) For a binary tree of height h
 - Maximum number of nodes = $2^{(h+1)} - 1$
 - Minimum number of nodes = $h + 1$
- (ii) For a complete binary tree of height h
 - Maximum number of nodes = $2^{(h+1)} - 1$
 - Minimum number of nodes = 2^h
- (iii) For a full binary tree :
Number of nodes = $2^{h+1} - 1$
- (iv) Total number of unlabelled binary trees with n nodes = $\frac{2^n C_n}{n+1}$
- (v) Total number of labelled binary tree with no keys = $\frac{2^n C_n}{n+1} \times n!$



(vi) Total number of binary trees with a given preorder (n length) = $\frac{2^n C_n}{n+1}$

(vii) Total number of binary trees with a given proorder/postorder/inorder = $\frac{2^n C_n}{n+1}$

(viii) Number of binary trees with a given preorder and inorder = 1

(ix) Number of binary trees with a given postorder and inorder = 1

(x) Number of leaf nodes = Number of nodes with two children + 1

8.2 Binary Tree Traversal

(1) Preorder :

- (i) Process the root.
- (ii) Traverse the left subtree of root in preorder.
- (iii) Traverse the right subtree of root in preorder.

(2) Inorder

- (i) Traverse the left subtree of root in inorder.
- (ii) Process the root.
- (iii) Traverse the right subtree of root in inorder.

(3) Post-order

- (i) Traverse the left subtree of root in post-order.
- (ii) Traverse the right subtree of root in post-order.
- (iii) Process the root.

In-order

• Morrison Traversal :

In-place traversal of a binary tree

No recursion

Constant extra space.

8.3 Binary Search Tree

A binary tree in which every node satisfy the property that all the keys in the left subtree of the node are smaller than node's key and all the keys in the right subtree of the node are greater than node's key.

- Inorder traversal of a binary search tree is ascending order of keys in the BST.
- Insertion in a BST
 - (a) $O(\log n)$ best case
 - (b) $O(n)$ worst case.

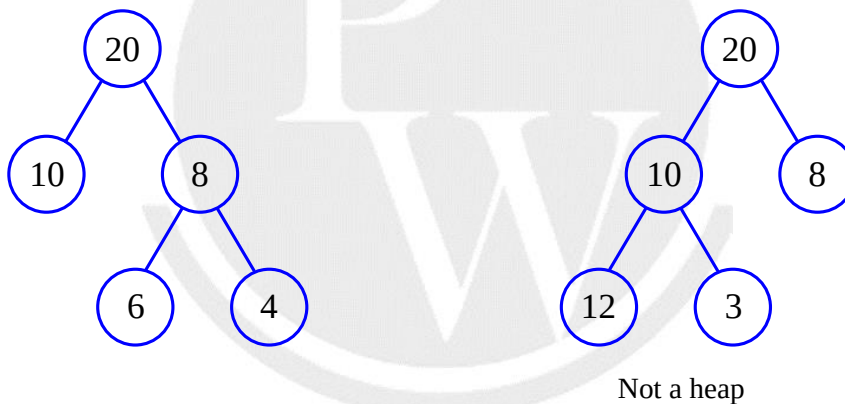
- Deletion from a BST :
 - Deletion of leaf node : can be done directly
 - Deletion of node with one child : copy the child to the node and delete the child.
 - Deletion of a node with two children : first find the inorder successor of the node, copy the contents of this successor to the node and delete the inorder successor.

We can also use inorder predecessor.
- TC : $O(\log n)$ best case
 $O(n)$ worst case

8.4 Heap

A complete binary tree in which every node satisfy heap property (min heap or max heap)

- Min heap : A complete binary tree in which every node satisfy the property that the value of the node is smaller than both its children.
- Max heap : A complete binary tree in which every node satisfy the property that the value in the node is greater than both its children.



- Construction of Heap :**
 - By inserting keys one after another : $O(n \log n)$
 - Using build-heap method/heapify algo : $O(n)$
- Number of min-heaps possible with n keys

$$T(n) = T(k) \cdot T(n - k - 1) \cdot {}^{n-1}C_k$$

K : Number of elements in the left subtree of root node.
- Deletion :
 - Swap $A[1], A[n]$
 - Apply Heapify on $A[1]$ considering there are only $(n - 1)$ nodes.

T.C = $O(\log n)$

- Heapsort (Assuming max heap)
 - Build Max Heap $O(n)$
 - Replace root element with last element ($A[1] \leftrightarrow A[n]$) reduce the heap size by 1 and then apply heapify at root node.
 - Repeat step 2 while heap size is greater than 1.

8.5 AVL Search Tree

- Height balanced BST.
- Balance factor (B_f) is given by

$$B_f = |h_L - h_R| \leq 1$$

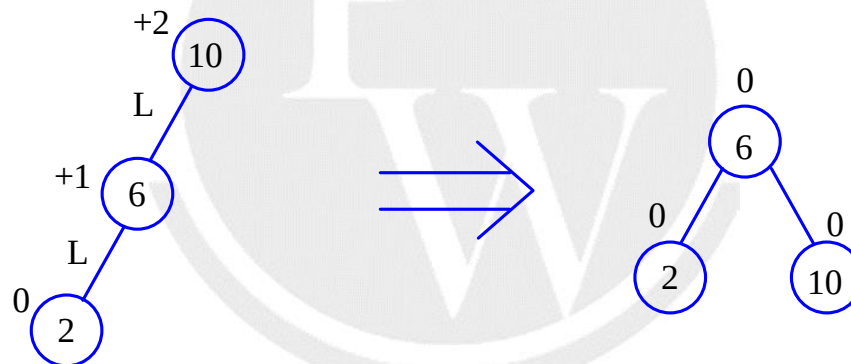
i.e., Balancing factor of a node can be +1, -1 or 0.

8.5.1 Insertion

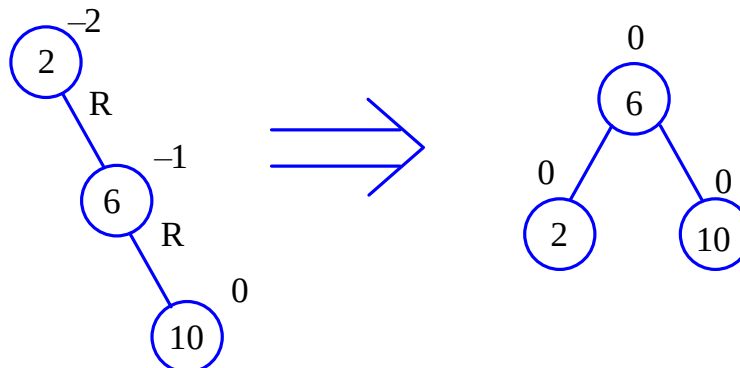
To ensure that the tree remains AVL tree, after insertion some re-balancing is performed i.e., certain rotations are needed.

8.5.2 Types of Rotation

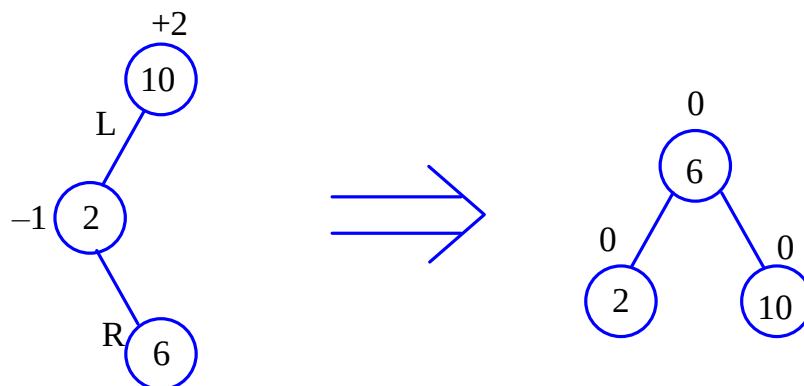
(i) Left-Left (LL) Rotation :



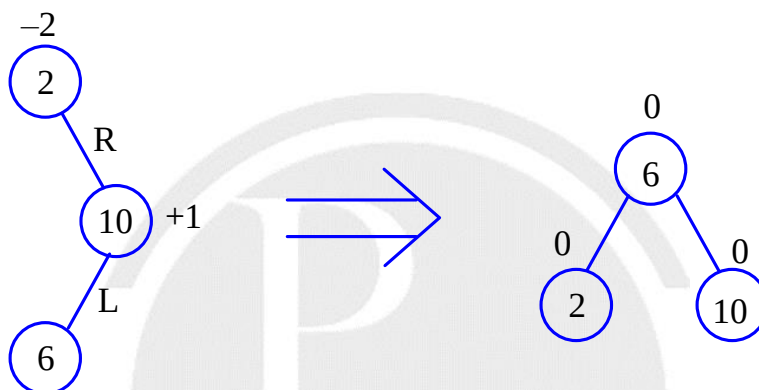
(ii) Right-Right (RR) Rotation :



(iii) Left-Right (LR) Rotation :



(iv) Right-Left (RL) Rotation :



- LL, RR are single rotation.
- LR, RL are double rotation.
- Just work no tri-node structure.
- Minimum number of nodes in an AVL tree of height h is given by recurrence.

$$n(h) = 1 + n(h-1) + n(h-2) \quad h \geq 2$$

$$n(0) = 1$$

$$n(1) = 2$$

i.e., it also forms a Fibonacci series

1, 2, 4, 7, 12,

